

[들어가기 앞서]

1. 문서를 작성하기 전, 다양한 방법을 시도해보았습니다.
2. Flag 형태를 제대로 확인하지 못해 정말 오래 걸렸습니다(Username과 password 말고 다른 것이 있는 줄 알았습니다)
3. Flag를 찾기까지의 과정을 작성해보도록 하겠습니다.

[어디부터 찾을 것인가?]

1. Android Manifest.xml
 2. MainActivity
 3. drawable
 4. layout
- 등이 있습니다.

[AndroidManifest.xml]

여기서 어떤 것이 'true'로 설정되어 있는지, 그리고 어떤 Activity들이 있는지 확인하고자 했습니다.

[1]

```
android:theme="@style/Theme.SolidApp"
android:label="@string/app_name"
android:icon="@drawable/logo"
android:allowBackup="true"
android:supportsRtl="true"
android:fullBackupContent="@xml/backup_rules"
android:roundIcon="@drawable/logo"
android:appComponentFactory="androidx.core.app.CoreComponentFactory"
android:dataExtractionRules="@xml/data_extraction_rules">
```

1. android:allowBackup="true"는 앱 데이터 백업을 허용하는 것
2. android:supportsRtl="true"는 RTL 언어를 지원한다는 것

[2]

```
<activity
  android:label="@string/title_activity_item_detail_host"
  android:name="com.ctf.solidapp.ui.login.ItemDetailHostActivity"
  android:exported="false"
  android:resizeableActivity="true">
  <meta-data
    android:name="android.app.lib_name"
    android:value="" />
</activity>
```

1. android:name="com.ctf.solidapp.ui.login.ItemDetailHostActivity" 이것은 앱이 실행 중일 때 실행하게 될 activity이다. 하지만, exported="false"이기 때문에, adb로 접근할 수는 없을 것이다.

2. android:resizeableActivity="true"는 화면 조절을 할 수 있다는 것이어서 공격에는 별 의미가 없을 것 같다.

[3]

```
<activity
  android:label="@string/app_name"
  android:name="com.ctf.solidapp.ui.login.LoginActivity"
  android:exported="true">
  <intent-filter>
    <action android:name="android.intent.action.MAIN" />
    <category android:name="android.intent.category.LAUNCHER" />
  </intent-filter>
</activity>
```

1. adb를 이용해서 am start -a android.intent.action.MAIN -n com.ctf.solidapp/ui.login.LoginActivity로 activity를 외부에서 실행해보았다.

2. 외부에서 접근할 수 있어서 설렘이나, 실행해보니 그냥 메인 로그인 화면으로 넘어가는 activity였고, MAIN과 LAUNCHER로 보아, 앱이 실행되면 먼저 실행되는 Activity이므로 문제가 없다.

3. 다만, Flag가 username과 password로 구성되어 있기 때문에, LoginActivity를 살펴볼 필요가 있다.

[4]

```
<receiver
  android:name="androidx.profileinstaller.ProfileInstallReceiver"
  android:permission="android.permission.DUMP"
  android:enabled="true"
  android:exported="true"
  android:directBootAware="false">
  <intent-filter>
    <action android:name="androidx.profileinstaller.action.INSTALL_PROFILE"/>
  </intent-filter>
  <intent-filter>
    <action android:name="androidx.profileinstaller.action.SKIP_FILE"/>
  </intent-filter>
  <intent-filter>
    <action android:name="androidx.profileinstaller.action.SAVE_PROFILE"/>
  </intent-filter>
  <intent-filter>
    <action android:name="androidx.profileinstaller.action.BENCHMARK_OPERATION"/>
  </intent-filter>
</receiver>
```

1. receiver에 원격 접속이 가능하게 설정돼 있다!
2. 그리고 intent-filter에 있는 신호를 보내게 되면 INSTALL_PROFILE, SKIP_FILE, SAVE_PROFILE, BENCHMARK_OPERATION 작업 중, 설정된 작업을 실행하게 된다.
3. 너무 의심스럽다. 그래서 androidx의 profileinstaller에서 ProfileInstallReceiver에 대해서 살펴볼 필요가 있을 것 같다.

[androidx.profileinstaller.ProfileInstallReceiver]

```
public void onReceive(Context context, Intent intent) {
    Bundle extras;
    if (intent == null) {
        return;
    }
    String action = intent.getAction();
    if (ACTION_INSTALL_PROFILE.equals(action)) {
        ProfileInstaller.writeProfile(context, new
ProfileInstallReceiver$$ExternalSyntheticLambda0(), new ResultDiagnostics(), true);
        return;
    }
    if (ACTION_SKIP_FILE.equals(action)) {
        Bundle extras2 = intent.getExtras();
        if (extras2 != null) {
            String string = extras2.getString(EXTRA_SKIP_FILE_OPERATION);
            if (EXTRA_SKIP_FILE_OPERATION_WRITE.equals(string)) {
                ProfileInstaller.writeSkipFile(context, new
ProfileInstallReceiver$$ExternalSyntheticLambda0(), new ResultDiagnostics());
                return;
            } else {
                if (EXTRA_SKIP_FILE_OPERATION_DELETE.equals(string)) {
                    ProfileInstaller.deleteSkipFile(context, new
ProfileInstallReceiver$$ExternalSyntheticLambda0(), new ResultDiagnostics());
                    return;
                }
                return;
            }
        }
        return;
    }
    if (ACTION_SAVE_PROFILE.equals(action)) {
        saveProfile(new ResultDiagnostics());
        return;
    }
    if (!ACTION_BENCHMARK_OPERATION.equals(action) || (extras = intent.getExtras()) == null) {
        return;
    }
    String string2 = extras.getString(EXTRA_BENCHMARK_OPERATION);
    ResultDiagnostics resultDiagnostics = new ResultDiagnostics();
    if (EXTRA_BENCHMARK_OPERATION_DROP_SHADER_CACHE.equals(string2)) {
        BenchmarkOperation.dropShaderCache(context, resultDiagnostics);
    } else {
        resultDiagnostics.onResultReceived(16, null);
    }
}
```

1. Broadcast를 통해서 action이 들어오면 action 신호에 맞는 작동을 하게 되는 것 같다.
2. 서칭을 해보니, am broadcast로 원하는 액션을 담아 임의로 브로드 캐스트를 발생시킬 수 있으며, 옵션으로 -n으로 특정 컴포넌트를 지정하면, 필터에 선언된 액션과 상관없이 해당 리시버가 무조건 메시지를 받게 된다고 한다. 이게 핵심이 아닐까?

3. 그래서 아래와 같은 브로드 캐스트를 발생시켜 보았다.

→ am broadcast -a androidx.profileinstaller.action.INSTALL_PROFILE -n
androidx.profileinstall/.ProfileInstallReceiver

→ am broadcast -a androidx.profileinstaller.action.SKIP_FILE -n
androidx.profileinstall/.ProfileInstallReceiver

→ am broadcast -a androidx.profileinstaller.action.SAVE_PROFILE -n
androidx.profileinstall/.ProfileInstallReceiver

→ am broadcast -a androidx.profileinstaller.action.BENCHMARK_OPERATION -n
androidx.profileinstall/.ProfileInstallReceiver

4. 다 해보았으나 변화가 없다. 이게 해결 방법이 아닌 것 같다.

[LoginActivity]

위에서 의심스러운 Activity 중 하나인, LoginActivity를 확인해봤습니다.

```
public static final void onCreate$lambda$5$lambda$4 (ProgressBar loading, LoginActivity this$0, EditText username, EditText password, View view) {
    Intrinsic.checkNotNullParameter (loading, "$loading" );
    Intrinsic.checkNotNullParameter (this$0, "this$0" );
    Intrinsic.checkNotNullParameter (username, "$username" );
    Intrinsic.checkNotNullParameter (password, "$password" );
    loading.setVisibility (0);
    LoginViewModel loginViewModel = this$0.loginViewModel ;
    if (loginViewModel == null) {
        Intrinsic.throwUninitializedPropertyAccessException ("LoginViewModel" );
        loginViewModel = null;
    }
    loginViewModel.login (username.getText ().toString (), password.getText ().toString ());
    AuthLogic authLogic = new AuthLogic ();
    if (authLogic.isPasswordValid (password.getText ().toString ()) & authLogic.isUsernameValid (username.getText ().toString ())) {
        Intent intent = new Intent (this$0, (Class<?>) ItemDetailHostActivity .class);
        Toast.makeText (this$0.getApplicationContext (), "Login Success!" , 0).show ();
        this$0.startActivity (intent);
    } else {
        Toast.makeText (this$0.getApplicationContext (), "Login Fail!" , 0).show ();
        this$0.finish ();
    }
}
```

AuthLogic을 이용해서 username과 password의 유효성을 확인하고 있는 것으로 보았고, 명확하게 보고자 했습니다.

[AuthLogic]

```
public final class AuthLogic {
    private final String decodedUsername = decodeBase64 ("cmVkdGVhbQ==" );
    private final String staticString = "gttcurct" ;
    private final int key = 6;

    private final String decodeBase64 (String encoded) {
        byte[] decodedBytes = Base64 .decode (encoded , 0);
        Intrinsic .checkNotNullExpressionValue (decodedBytes , "decodedBytes" );
        return new String (decodedBytes , Charsets .UTF_8 );
    }

    private final String xorString (String input , int key) {
        StringBuilder sb = new StringBuilder ();
        int length = input .length ();
        for (int i = 0; i < length ; i++) {
            sb.append ((char) (input .charAt (i) ^ key));
        }
        String sb2 = sb.toString ();
        Intrinsic .checkNotNullExpressionValue (sb2 , "xorResult.toString()" );
        return sb2;
    }

    public final boolean isUsernameValid (String inputUsername ) {
        Intrinsic .checkNotNullParameter (inputUsername , "inputUsername" );
        return Intrinsic .areEqual (inputUsername , this .decodedUsername );
    }

    public final boolean isPasswordValid (String inputPassword ) {
        Intrinsic .checkNotNullParameter (inputPassword , "inputPassword" );
        return Intrinsic .areEqual (inputPassword , xorString (this .staticString , this .key));
    }
}
```

1. private final String decodedUsername = decodeBase64("cmVkdGVhbQ==")를 보고 username은 cmVkdGVhbQ==을 Base64로 디코딩해주면 될 것이라고 예상했습니다.

Base64 형식에서 디코딩

데이터를 입력하고 디코딩 버튼을 누르:

cmVkdGVhbQ=

< 디코딩 >

redteam

2. String xorString 함수를 보니 xor string이라는 암호화 방법을 이용한 것 같아서 xor string decoder를 이용하면 될 것이라 생각했습니다.

gttcurct

arrester

6

Input Type Key Type Output Type

Text Hexadecimal Text

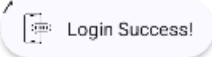
Xor

Static string 값과, key 값을 이용해 디코딩했습니다.

이것으로 로그인을 시도해보니

13 Item 13

14 Item 14



15 Item 15

성공했습니다!

[결과]

ItemListFragment	
1	Item 1
2	Item 2
3	Item 3
4	Item 4
5	Item 5
6	Item 6
7	Item 7
8	Item 8
9	Item 9
10	Item 10
11	Item 11
12	Item 12
13	Item 13
14	Item 14
15	Item 15

SolidApp **Solved**

[Korean] 사내 공지 앱의 기능 테스트를 위해 일부 기능만 포함된 버전으로 앱 점검을 요청합니다. [English] To test the functionality of the company's notice app, we request an app inspection with a version that includes only some of the features. flag format: flag{Username_Password}

100 points Android Ko

Private Block App

Is this really a secure p

500 points Android Ko

SolidApp ✕

[Korean] 사내 공지 앱의 기능 테스트를 위해 일부 기능만 포함된 버전으로 앱 점검을 요청합니다. [English] To test the functionality of the company's notice app, we request an app inspection with a version that includes only some of the features. flag format: flag{Username_Password}

[Download](#)

이미 해결한 문제입니다!

[과제 후기]

사실 초등학생 때는 모바일에 대해서 관심이 많았습니다. 스마트폰을 루팅해보고 변화된 값을 찾아서 게임 재화량을 바꿔보기도 하고, APK도 만들어보고 싶었습니다.

그 때 당시 궁금했던 점들을 드디어 지금에서야 어느정도 해소해볼 수 있었습니다. 비록 플래그 양식을 몰라서, FLAG를 찾기도 삽질(?)로 많은 시간을 소모하긴 했지만, 그래도 많은 걸 얻어갑니다. 개인적으로 어렸을 때 생각도 나고 기분이 참 오묘했습니다. 고맙습니다!